Problem Set 1 — Big-O from Pseudocode

VCE Algorithmics (HESS) · Unit 4 AOS1 · Week 1

Due: start of Week 2. Test 1 (SAT Criterion 5) is Friday 24 July.

Name: _____ **Due:** Monday 20 July

Method reminder: identify the input size n → count loop iterations → nesting multiplies, sequence adds → keep the dominant term → state the *tightest* bound and justify it in one sentence.

Section A — Warm-up (growth classes)

A1. Order these from slowest-growing to fastest-growing:

$$n^2, \quad \log n, \quad n!, \quad n \log n, \quad 1, \quad 2^n, \quad n, \quad n^3$$

A2. Simplify each to tightest Big-O form:

a.) $O(4n^2 + 100n + 7)$

b.) $O(n \log n + n^2)$

c.) $O(2^n + n^{50})$

d.) $O(\frac{n(n-1)}{2})$

e.) $O(3 \log n + 8)$

f.) $O(n+m)$ where m is a second input size — careful!

A3. Using the formal definition of Big-O, show that $f(n) = 6n + 14$ is $O(n)$ by finding constants c and n_0 such that $f(n) \leq c \cdot n$ for all $n \geq n_0$.

Section B — Counting operations

B1. For the algorithm below, let n be the length of `L`.

```
Algorithm SumAndScale(L, k):
    total ← 0
    Foreach x in L Do
        total ← total + x
    total ← total * k
    Return total
```

a.) Count the exact number of assignments and additions as a function of n.

b.) State the tightest Big-O bound.

B2. A student claims the following algorithm is $O(n^2)$ "because there are two loops". The claim is *wrong*. Give the correct complexity and explain the student's error.

```
Algorithm TwoLoops(L):
    Foreach x in L Do
        print(x)
    Foreach x in L Do
        print(x * x)
```

Section C — Big-O from pseudocode

For each algorithm, state the **worst-case** time complexity in tightest Big-O form, with a one-sentence justification. n is the length of the input list unless stated.

C1.

```
Algorithm ContainsNegative(L):
    Foreach x in L Do
        If x < 0 Do
            Return True
    Return False
```

C2.

```
Algorithm CommonElement(A, B):
    Foreach a in A Do
        Foreach b in B Do
            If a = b Do
                Return True
    Return False
```

Let $|A| = n$ *and* $|B| = m$.

C3.

```
Algorithm Halver(n):
    count ← 0
    While n > 1 Do
        n ← n div 2
        count ← count + 1
    Return count
```

C4.

```
Algorithm TriangleSum(L):
    total ← 0
    Foreach i from 1 To n Do
        Foreach j from i To n Do
            total ← total + L[i] * L[j]
    Return total
```

C5.

```
Algorithm StrangePair(L):
    Foreach i from 1 To n Do
        j ← n
        While j   1 Do
            print(L[i], L[j])
            j ← j div 2
    Return
```

C6.

```
Algorithm GridSearch(G):
    Foreach node v in G Do
        Foreach neighbour u of v Do
            If label(u) = label(v) Do
                Return True
    Return False
```

Let G have $|V|$ nodes and $|E|$ edges. Hint: how many times is the inner body executed ***in total across the whole run****, not per node?*

Section D — Best and worst case

D1. For `ContainsNegative` (C1), describe a best-case input and a worst-case input, and give the complexity of each.

D2. The early-exit Bubble Sort from class is $O(n^2)$ and $\Omega(n)$, but a student writes it is "$\Theta(n^2)$". Explain precisely why the Θ claim fails when we quantify over *all* inputs.

Section E — Challenge

E1. Determine the worst-case complexity, with justification:

```
Algorithm Harmonic(L):
    Foreach i from 1 To n Do
        j ← i
        While j   n Do
            print(L[j])
            j ← j + i
```

Hint: for each i, the inner loop runs about n/i times. What is $\sum_{i=1}^{n} \frac{n}{i}$ approximately? You may use the fact that $1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \approx \ln n$.

E2. Your SAT solution from Unit 3 almost certainly contains at least one loop over your graph's nodes or edges. Choose one module of your own solution, write down its loop structure (you may paraphrase), and give a first estimate of its worst-case complexity. *This is a head start on Memo 02.*

Bring your attempt to the first lesson of Week 2 — we will mark Section C together before Test 1 revision.